МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность

Образовательная программа: «Технологии защиты информации»

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Реализация уровня приложения с возможностью многопользовательского доступа к созданной БД»

Выполнил студент(ы):

группа.поток ИББД 1.7

Пахомов Владимир Юрьевич / 

ФИО *Подпись*

Сивков Иван Александрович / 

ФИО *Подпись*

Булин Михаил Михайлович / 

ФИО *Подпись*

Проверил:

Ярцева Наталия Андреевна / _____

ФИО *Подпись*

_____

Отметка о выполнении (один из вариантов:

отлично, хорошо, удовлетворительно, зачтено)

_____

Дата

Санкт-Петербург

2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Выбор стека технологий и архитектура системы **4**
1.1 Обоснование серверного стека (FastAPI, SQLAlchemy 2.0, Psycopg 3) 4
1.2 Обоснование клиентского стека (React 19, TypeScript, Tailwind CSS) 4
1.3 Общая архитектура и интеграция с механизмами защиты СУБД 4

2 Объектно-реляционное отображение (ORM SQLAlchemy) **5**
2.1 Маппинг базовых сущностей предметной области 5
2.2 Маппинг представлений (VIEW) 6

3 Реализация ролевого многопользовательского доступа **7**
3.1 Матрица разграничения доступа 7
3.2 Реализация контроля доступа в модуле app.access 7

4 Сложные аналитические запросы с группировкой (GROUP BY) **8**
4.1 Аналитический отчет для роли «Клиент» 8
4.2 Аналитический отчет для роли «Диспетчер флота» 8
4.3 Аналитический отчет для роли «Администратор» 8

5 Демонстрация работы пользовательского интерфейса **10**
5.1 Аутентификация и вход под разными ролями 10
5.2 Ограничение списка таблиц в зависимости от учетной записи 10
5.3 Выполнение операций CRUD и отражение в триггерах аудита 11

6 Листинги ключевых модулей системы **12**
6.1 Контроллер таблиц и CRUD операций (backend/app/routers/tables.py) 12
6.2 Модуль отчетов со сложными запросами (backend/app/routers/reports.py) 13
6.3 Главный интерфейсный модуль React (frontend/src/pages/main.tsx) 14

ЗАКЛЮЧЕНИЕ **16**

ВВЕДЕНИЕ

Цель работы: получение практических навыков использования серверных языков программирования, современных объектно-реляционных отображений (ORM), создания защищенного программного интерфейса приложения (API) и веб-интерфейса, обеспечивающего ролевой многопользовательский доступ к реляционной базе данных с соблюдением принципов информационной безопасности.

В рамках лабораторной работы создано полноценное клиент-серверное программное обеспечение для автоматизированной информационной системы морских контейнерных перевозок, база данных которой спроектирована и развернута в лабораторных работах №1–2. Архитектура программного решения включает:

- **Серверную часть (Backend)**: на базе асинхронного веб-фреймворка FastAPI, ORM SQLAlchemy и драйвера Psycopg, реализующую защищенный REST API с токен-ориентированной JWT-аутентификацией;
- **Клиентскую часть (Frontend)**: на базе стека React, TypeScript, Vite и Tailwind CSS, предоставляющую динамический адаптивный веб-интерфейс в соответствии с назначенной пользователю ролью;
- **Слой информационной безопасности и ролевого разграничения**: изоляцию данных грузоотправителей, предоставление специализированных представлений (*VIEW*) и агрегированных аналитических отчетов со сложными соединениями (*JOIN*) и группировками (*GROUP BY*).

1 ВЫБОР СТЕКА ТЕХНОЛОГИЙ И АРХИТЕКТУРА СИСТЕМЫ

1.1 Обоснование серверного стека (FastAPI, SQLAlchemy 2.0, Psycopg 3)

Для реализации прикладного уровня выбрана связка технологий на языке Python:

- **Язык программирования Python 3.14** обеспечивает высокую скорость и безопасность разработки, прозрачность синтаксиса и обширную экосистему библиотек для работы с базами данных и криптографией.
- **Фреймворк FastAPI**: высокопроизводительный асинхронный фреймворк, базирующийся на спецификации ASGI и стандарте OpenAPI/Swagger. Обеспечивает автоматическую строгую валидацию входных/выходных структур данных посредством Pydantic, что предотвращает атаки внедрения некорректных типов данных.
- **ORM SQLAlchemy 2.0**: промышленный стандарт объектно-реляционного маппинга. Использование современной типизированной декларативной модели (`Mapped`, `mapped_column`) полностью исключает риск SQL-инъекций (SQL Injection), так как все формируемые запросы параметризуются на уровне драйвера СУБД.
- **Драйвер Psycopg 3**: современная реализация протокола клиентского взаимодействия с PostgreSQL, поддерживающая расширенные типы данных и высокую скорость выполнения транзакций.

1.2 Обоснование клиентского стека (React 19, TypeScript, Tailwind CSS)

В качестве платформы пользовательского интерфейса выбран компонентный SPA-подход (Single Page Application):

- **React 19 и TypeScript**: обеспечивают строгую статическую типизацию на клиенте, предотвращая логические ошибки рендеринга и работы с состоянием аутентификации.
- **Vite**: современный инструмент сборки frontend-приложений, обеспечивающий высокую скорость запуска и оптимизацию результирующего бандла.
- **Tailwind CSS v4**: утилитарный CSS-фреймворк для быстрой верстки чистого, адаптивного интерфейса системы с поддержкой темного и светлого режимов отображения.

1.3 Общая архитектура и интеграция с механизмами защиты СУБД

На рисунке 1 представлена трехуровневая архитектура информационной системы.

При разработке архитектуры заложен принцип сквозной информационной безопасности:

1. На уровне приложения аутентификация осуществляется по стандарту OAuth2 с выдачей JWT-токенов (алгоритм HS256). В токене фиксируется имя субъекта и его системная роль.
2. Матрица доступа приложения (`app.access`) строго ограничивает перечень доступных таблиц и допустимых операций (`read`, `write`, `delete`) для каждой роли.
3. Взаимодействие с базой данных спроектировано с учетом требований ЛР3:
    - Любые операции модификации (`POST`, `PATCH`, `DELETE`), инициируемые через API, приводят к выполнению SQL-команд в PostgreSQL, на которые навешаны триггеры логирования, автоматически фиксирующие пользователя, время и дельту данных в служебной таблице аудита.
    - Конфиденциальные таблицы (таблица секретных ключей и токенов шифрования, исследуемая в рамках симметричного шифрования pgcrypto) исключены из реестра доступных клиентам и диспетчерам отношений, обеспечивая соблюдение принципа наименьших привилегий (PoLP).

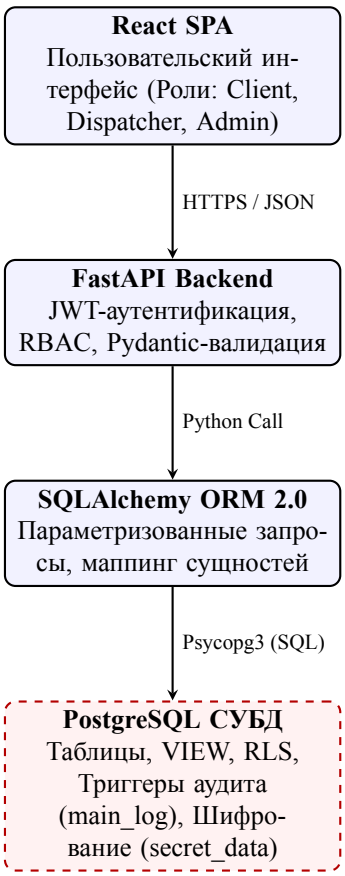


Рисунок 1 — Трехуровневая архитектура информационной системы

2 ОБЪЕКТНО-РЕЛЯЦИОННОЕ ОТОБРАЖЕНИЕ (ORM SQLALCHEMY)

В модулях пакета `app.models` реализовано строго типизированное отображение реляционных таблиц и представлений БД в классы SQLAlchemy 2.0 с использованием типизации `Mapped` и `mapped_column`.

2.1 Маппинг базовых сущностей предметной области

Ниже приведен фрагмент декларативного описания сущностей «Заказ» (*Order*) и «Рейс» (*Voyage*) с сохранением целостности связей и проверочных ограничений (*CHECK*), установленных в ЛР2:

```
from datetime import datetime
from decimal import Decimal
from typing import Optional
from sqlalchemy import Integer, String, Numeric, DateTime, ForeignKey, CheckConstraint, func
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.models.base import Base

class Order(Base):
    __tablename__ = "orders"
    __table_args__ = (
        CheckConstraint("cargo_weight > 0", name="chk_orders_cargo_weight"),
        CheckConstraint("cargo_volume > 0", name="chk_orders_cargo_volume"),
        CheckConstraint("total_price >= 0", name="chk_orders_total_price"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    client_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("clients.id", onupdate="CASCADE", ondelete="RESTRICT"), nullable=False
    )
    cargo_name: Mapped[str] = mapped_column(String(150), nullable=False)
```

```
    cargo_weight: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    cargo_volume: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    total_price: Mapped[Decimal] = mapped_column(Numeric(12, 2), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now(), nullable=False
    )

    # Отношения к смежным сущностям
    client: Mapped["Client"] = relationship(back_populates="orders")
    invoice: Mapped[Optional["Invoice"]] = relationship(
        back_populates="order", uselist=False, cascade="all, delete-orphan", passive_deletes=True
    )
    order_containers: Mapped[list["OrderContainer"]] = relationship(
        back_populates="order", cascade="all, delete-orphan", passive_deletes=True
    )
    voyage_orders: Mapped[list["VoyageOrder"]] = relationship(
        back_populates="order", cascade="all, delete-orphan", passive_deletes=True
    )
```

2.2 Маппинг представлений (VIEW)

Для работы с представлениями, созданными в ЛР1 и ЛР2, классы ORM сконфигурированы с метапараметром `{"info": {"is_view": True}}`, что предотвращает попытки создания физических таблиц миграциями и информирует систему о режиме «только чтение»:

```
class ViewClientTracking(Base):
    __tablename__ = "view_client_tracking"
    __table_args__ = {"info": {"is_view": True}}

    order_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    client_id: Mapped[int] = mapped_column(Integer)
    cargo_name: Mapped[str] = mapped_column(String(150))
    vessel_name: Mapped[str] = mapped_column(String(100))
    port_of_loading: Mapped[str] = mapped_column(String(100))
    port_of_discharge: Mapped[str] = mapped_column(String(100))
    departure_date: Mapped[datetime] = mapped_column(DateTime)
    arrival_date: Mapped[datetime] = mapped_column(DateTime)
    voyage_status: Mapped[str] = mapped_column(String(30))

class ViewVoyageManifest(Base):
    __tablename__ = "view_voyage_manifest"
    __table_args__ = {"info": {"is_view": True}}

    voyage_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    ship_name: Mapped[str] = mapped_column(String(100))
    ship_capacity_tons: Mapped[Decimal] = mapped_column(Numeric(10, 2))
    origin_port: Mapped[str] = mapped_column(String(100))
    destination_port: Mapped[str] = mapped_column(String(100))
    total_orders_assigned: Mapped[int] = mapped_column(Integer)
    total_cargo_tons: Mapped[Decimal] = mapped_column(Numeric(10, 2))
```

3 РЕАЛИЗАЦИЯ РОЛЕВОГО МНОГОПОЛЬЗОВАТЕЛЬСКОГО ДОСТУПА

3.1 Матрица разграничения доступа

В соответствии с концептуальным проектом выделены 3 основные роли пользователей:

1. **Клиент (*client*)** — грузоотправитель, имеющий доступ исключительно к своим заказам, счетам, трекингу грузов и представлению контейнеров своего заказа. Разрешено создание новых заказов. Доступ к чужим заказам блокируется на уровне фильтрации внешнего ключа `client_id`.
2. **Диспетчер флота (*dispatcher*)** — сотрудник логистического центра порта. Имеет доступ к судовому грузовому манифесту, ведомостям безопасности пломб, реестрам судов, портов, контейнеров и рейсов. Обладает полномочиями планирования рейсов (`write` в *voyages* и *voyage_orders*).
3. **Администратор безопасности (*admin*)** — обладает полным доступом (`CRUD`) ко всем сущностям и служебным таблицам базы данных.

Сводная матрица полномочий представлена в таблице 1.

Таблица 1 — Матрица полномочий ролевой модели доступа

Роль	Чтение (SELECT / READ)	Создание/Изменение (WRITE)	Удаление (DELETE)
client	`view_client_orders`, `view_client_tracking`, `orders`, `invoices`, `order_containers`, `clients`	`orders`	Запрещено
dispatcher	`view_voyage_manifest`, `view_container_safety_audit`, `voyages`, `ships`, `ports`, `containers`, `voyage_orders`	`voyages`, `voyage_orders`	Запрещено
admin	Все таблицы и представления	Все физические таблицы	Все физические таблицы

3.2 Реализация контроля доступа в модуле app.access

Разграничение прав реализовано централизованно в программном слое backend:

```
ROLE_ACCESS = {
    "client": {
        "read": [
            "view_client_orders", "view_client_tracking", "orders",
            "invoices", "order_containers", "clients"
        ],
        "write": ["orders"],
        "delete": [],
    },
    "dispatcher": {
        "read": [
            "view_voyage_manifest", "view_container_safety_audit", "voyages",
            "ships", "ports", "containers", "voyage_orders"
        ],
        "write": ["voyages", "voyage_orders"],
        "delete": [],
    },
    "admin": {
        "read": list(TABLE_REGISTRY),
        "write": list(DOMAIN_TABLES),
        "delete": list(DOMAIN_TABLES),
    },
}
```

При попытке выполнения операции обращения к таблице или строке зависимость `get_current_user` проверяет валидность токена, после чего эндпоинт валидирует право доступа. В случае отсутствия прав клиенту

возвращается статус `HTTP 403 Forbidden`.

4 СЛОЖНЫЕ АНАЛИТИЧЕСКИЕ ЗАПРОСЫ С ГРУППИРОВКОЙ (GROUP BY)

В соответствии с пунктом 4 задания для каждой выделенной группы пользователей разработан специализированный эндпоинт в модуле `app.routers.reports`, выполняющий сложный агрегационный запрос к более чем двум сущностям с использованием соединений `JOIN`, группировки `GROUP BY` и возможностью сортировки и фильтрации.

4.1 Аналитический отчет для роли «Клиент»

Запрос выполняет расчет сводной финансовой статистики и объемов перевозок клиента по статусам оплаты счетов. Задействованы сущности: *clients*, *orders*, *invoices*.

```
SELECT
    c.name AS client_name,
    COALESCE(i.payment_status, 'No Invoice') AS invoice_status,
    COUNT(o.id) AS total_orders,
    COALESCE(SUM(o.cargo_weight), 0) AS total_cargo_weight,
    COALESCE(SUM(o.total_price), 0) AS total_declared_price,
    COALESCE(SUM(i.amount), 0) AS total_invoiced_amount
FROM clients c
JOIN orders o ON c.id = o.client_id
LEFT JOIN invoices i ON o.id = i.order_id
WHERE c.id = :client_id
GROUP BY c.name, COALESCE(i.payment_status, 'No Invoice')
ORDER BY total_declared_price DESC;
```

4.2 Аналитический отчет для роли «Диспетчер флота»

Запрос формирует сводную загрузку портов отправления: число выполненных рейсов, суммарный дедвейт задействованных контейнеровозов и общее количество обработанных заказов. Задействованы сущности: *ports*, *voyages*, *ships*, *voyage_orders*.

```
SELECT
    p.name AS port_name,
    p.country AS country,
    COUNT(DISTINCT v.id) AS total_departures,
    COALESCE(SUM(s.deadweight), 0) AS total_deadweight_serviced,
    COUNT(vo.order_id) AS total_orders_routed
FROM ports p
JOIN voyages v ON p.id = v.departure_port_id
JOIN ships s ON v.ship_id = s.id
LEFT JOIN voyage_orders vo ON v.id = vo.voyage_id
GROUP BY p.name, p.country
HAVING COUNT(DISTINCT v.id) > 0
ORDER BY total_deadweight_serviced DESC;
```

4.3 Аналитический отчет для роли «Администратор»

Глобальный мониторинг использования флота по типам и флагам приписки судов с подсчетом перевезенных объемов и выручки. Задействованы сущности: *ships*, *voyages*, *voyage_orders*, *orders*.

```
SELECT
    s.flag_state,
    s.name AS ship_name,
    COUNT(DISTINCT v.id) AS total_voyages,
    COUNT(vo.order_id) AS total_orders_loaded,
```

```sql
    COALESCE(SUM(o.cargo_weight), 0) AS total_weight_carried,
    COALESCE(SUM(o.total_price), 0) AS gross_revenue
FROM ships s
LEFT JOIN voyages v ON s.id = v.ship_id
LEFT JOIN voyage_orders vo ON v.id = vo.voyage_id
LEFT JOIN orders o ON vo.order_id = o.id
GROUP BY s.flag_state, s.name
ORDER BY gross_revenue DESC;
```

5 ДЕМОНСТРАЦИЯ РАБОТЫ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

5.1 Аутентификация и вход под разными ролями

Для подтверждения многопользовательского режима в базу данных добавлены преднастроенные учетные записи:

- `balttrade` (пароль: `client123`, роль: *client*, привязан к *client_id = 1*);
- `dispatcher` (пароль: `dispatcher123`, роль: *dispatcher*);
- `admin` (пароль: `admin123`, роль: *admin*).

Форма авторизации (`/login`) передает учетные данные в формате `application/x-www-form-urlencoded` на эндпоинт `/api/v1/login`, получая подписанный JWT токен.

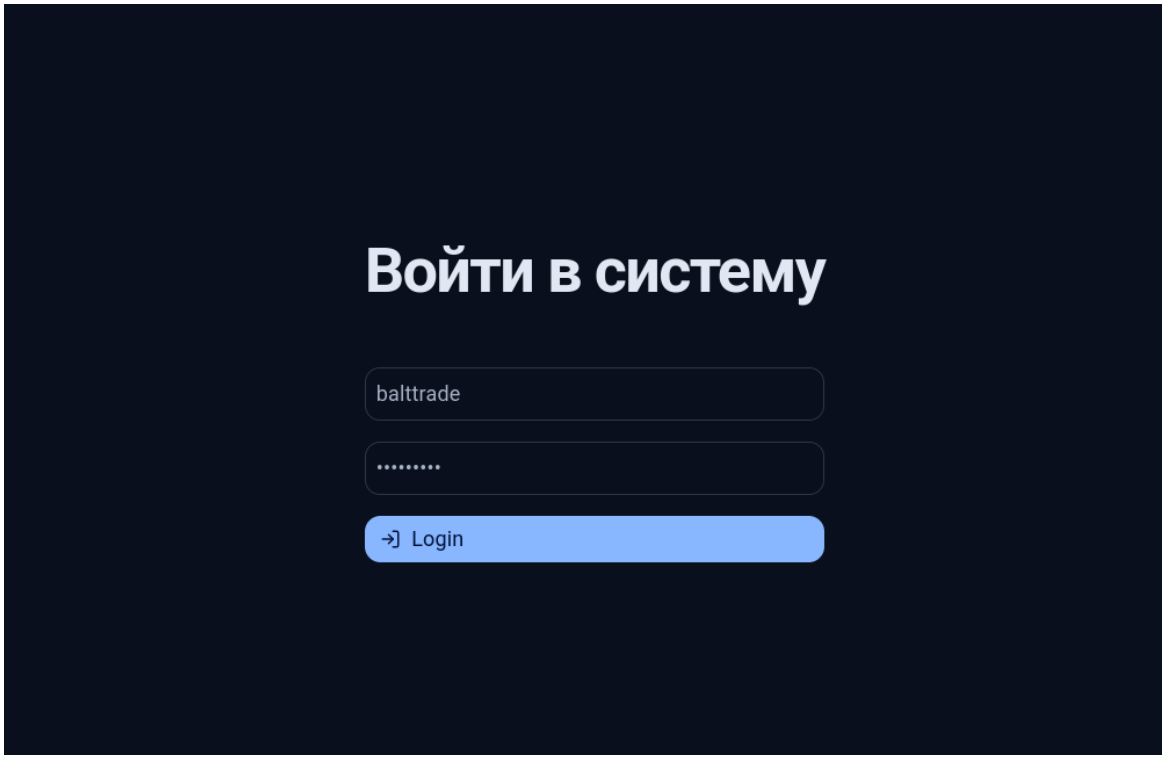


Рисунок 2 — Окно аутентификации в пользовательском интерфейсе

5.2 Ограничение списка таблиц в зависимости от учетной записи

После успешной аутентификации клиент запрашивает профиль пользователя через `/api/v1/me` и динамический список доступных таблиц через `/api/v1/tables`:

- Пользователю `balttrade` доступны только 6 объектов: представления `view_client_orders`, `view_client_tracking` и таблицы `orders`, `invoices`, `order_containers`, `clients`. Кнопка добавления записи активна только для таблицы `orders`.
- Диспетчеру доступны технологические представления манифеста и таблицы рейсов, судов, портов и контейнеров.
- Администратор видит полный реестр объектов с включенной возможностью редактирования и удаления.

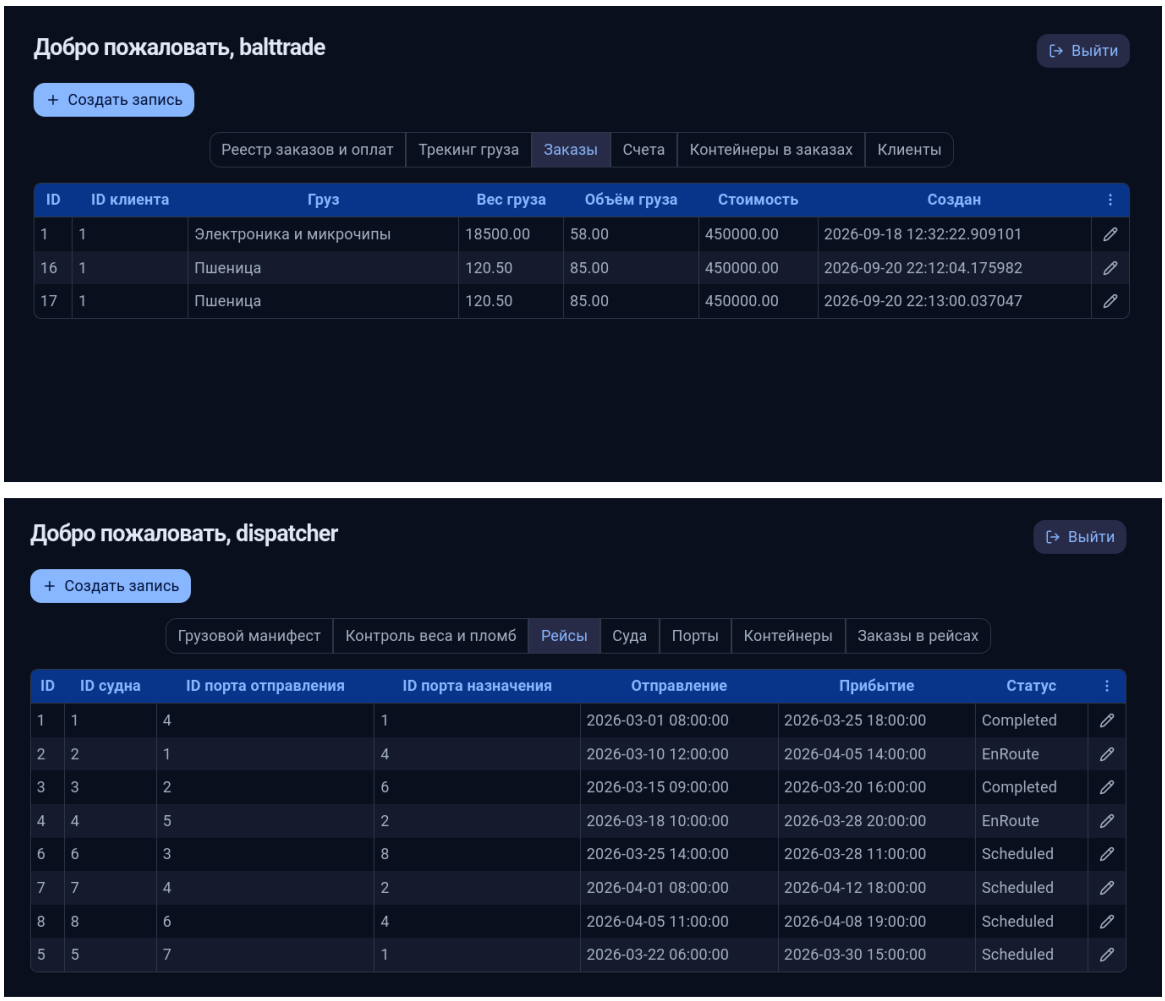


Рисунок 3 — Отображение доступных таблиц в пользовательском интерфейсе в зависимости от роли пользователя

5.3 Выполнение операций CRUD и отражение в триггерах аудита

При создании нового заказа клиентом (рисунок 4) отправляется запрос `POST /api/v1/tables/orders`. Вспомогательный обработчик backend автоматически подставляет авторизованный `client_id`, исключая возможность фальсификации отправителя.

На уровне PostgreSQL в результате данной транзакции срабатывает триггер мониторинга БД, вносящий в таблицу логов `main_log` запись с типом `'I'`, меткой `now()`, текущим пользователем и JSON-представлением созданного кортежа.

При попытке несанкционированного доступа (например, запросе диспетчером финансовых счетов клиентов через `GET /api/v1/tables/invoices`) сервер возвращает:

```
HTTP/1.1 403 Forbidden
{"detail": "Недостаточно прав для чтения таблицы invoices"}
```

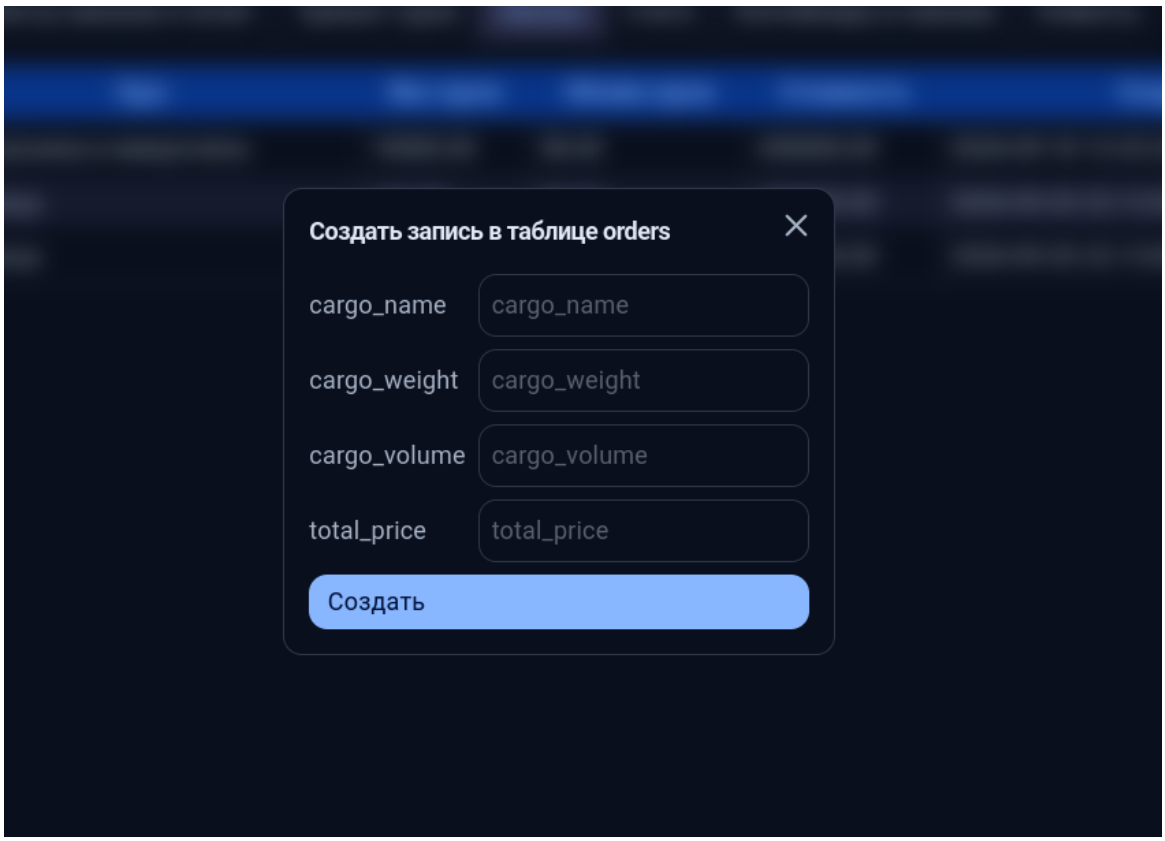


Рисунок 4 — Модальное окно создания заказа в пользовательском интерфейсе

6 ЛИСТИНГИ КЛЮЧЕВЫХ МОДУЛЕЙ СИСТЕМЫ

6.1 Контроллер таблиц и CRUD операций (backend/app/routers/tables.py)

```
from typing import Any
from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy import desc, select
from sqlalchemy.orm import Session

from app.access import can_delete, can_read, can_write, is_view, readable_tables
from app.db import get_db
from app.deps import get_current_user
from app.models import AppUser, Order
from app.schemas import RowPayload, TableListResponse
from app.tables import get_model, serialize_row

router = APIRouter(tags=["tables"])

@router.get("/tables")
def list_available_tables(user: AppUser = Depends(get_current_user)) -> dict[str, list[dict]]:
    tables = []
    for name in readable_tables(user.role):
        tables.append({
            "name": name,
            "kind": "view" if is_view(name) else "table",
            "can_write": can_write(user.role, name),
            "can_delete": can_delete(user.role, name),
        })
    return {"tables": tables}

@router.get("/tables/{table_name}", response_model=TableListResponse)
```

```python
def read_table_data(
    table_name: str,
    limit: int = Query(default=100, ge=1, le=1000),
    user: AppUser = Depends(get_current_user),
    db: Session = Depends(get_db),
) -> TableListResponse:
    if not can_read(user.role, table_name):
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Доступ запрещен")

    model = get_model(table_name)
    stmt = select(model)
    # Фильтрация контекста клиента
    if user.role == "client":
        if hasattr(model, "client_id") and user.client_id:
            stmt = stmt.where(model.client_id == user.client_id)
        elif table_name == "clients" and user.client_id:
            stmt = stmt.where(model.id == user.client_id)

    rows = db.scalars(stmt.limit(limit)).all()
    return TableListResponse(table=table_name, items=[serialize_row(r) for r in rows], count=len(
    rows))

@router.post("/tables/{table_name}", status_code=status.HTTP_201_CREATED)
def create_row(
    table_name: str,
    payload: RowPayload,
    user: AppUser = Depends(get_current_user),
    db: Session = Depends(get_db),
) -> dict:
    if not can_write(user.role, table_name) or is_view(table_name):
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Запись запрещена")

    model = get_model(table_name)
    data = payload.model_dump()
    if user.role == "client" and table_name == "orders":
        data["client_id"] = user.client_id

    instance = model(**data)
    db.add(instance)
    db.commit()
    db.refresh(instance)
    return serialize_row(instance)
```

6.2 Модуль отчетов со сложными запросами (backend/app/routers/reports.py)

```python
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy import desc, func, select
from sqlalchemy.orm import Session
from app.db import get_db
from app.deps import get_current_user
from app.models import AppUser, Client, Invoice, Order, Port, Ship, Voyage, VoyageOrder
from app.schemas import ReportResponse

router = APIRouter(prefix="/reports", tags=["reports"])

@router.get("/client", response_model=ReportResponse)
def get_client_report(user: AppUser = Depends(get_current_user), db: Session = Depends(get_db)):
    if user.role != "client" or not user.client_id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Доступно только клиенту")
```

```
    stmt = (
        select(
            Client.name.label("client_name"),
            func.coalesce(Invoice.payment_status, "Не выставлен").label("payment_status"),
            func.count(Order.id).label("total_orders"),
            func.coalesce(func.sum(Order.cargo_weight), 0).label("total_weight"),
            func.coalesce(func.sum(Order.total_price), 0).label("total_price"),
        )
        .join(Order, Client.id == Order.client_id)
        .outerjoin(Invoice, Order.id == Invoice.order_id)
        .where(Client.id == user.client_id)
        .group_by(Client.name, func.coalesce(Invoice.payment_status, "Не выставлен"))
        .order_by(desc("total_price"))
    )
    res = db.execute(stmt).mappings().all()
    return ReportResponse(
        role="client",
        title="Сводная статистика заказов клиента",
        description="Группировка заказов по статусам оплаты",
        items=[dict(r) for r in res],
        count=len(res)
    )
```

6.3 Главный интерфейсный модуль React (frontend/src/pages/main.tsx)

```
import { useEffect, useState, useCallback } from "react";
import { Navigate, useNavigate } from "react-router";
import { LogOut, Plus } from "lucide-react";
import { RowDialog } from "../components/row-dialog";
import { TableLayout } from "../components/table-layout";
import { TablesBar } from "../components/tables-bar";
import type { TableInfo, Table, User } from "../types/types";
import { fetchTables, fetchTable, createRow, deleteRow, updateRow } from "../utils/tables";

export function Main() {
  const navigate = useNavigate();
  const [token, setToken] = useState(localStorage.getItem("token"));
  const [rowDialogOpen, setRowDialogOpen] = useState(false);
  const [user, setUser] = useState<User | null>(null);
  const [tablesInfo, setTablesInfo] = useState<TableInfo[]>([]);
  const [selectedTable, setSelectedTable] = useState<Table | null>(null);

  const getTable = useCallback(async (tableName: string) => {
    if (!token) return;
    const data = await fetchTable(token, tableName);
    const info = tablesInfo.find((t) => t.name === tableName);
    setSelectedTable({
      name: tableName,
      kind: info?.kind ?? "table",
      can_write: info?.can_write ?? false,
      can_delete: info?.can_delete ?? false,
      count: data.count,
      items: data.items,
    });
  }, [token, tablesInfo]);

  useEffect(() => {
    if (!token) return;
    async function load() {
      const meRes = await fetch("/api/v1/me", { headers: { Authorization: `Bearer ${token}` } });
```

```
      if (!meRes.ok) { localStorage.removeItem("token"); setToken(null); return; }
      setUser(await meRes.json());
      setTablesInfo(await fetchTables(token!));
    }
    void load();
  }, [token]);

  if (!token) return <Navigate to="/login" replace />;

  return (
    <main className="max-w-6xl mx-auto pt-16">
      <div className="flex justify-between items-center">
        <h2Добро> пожаловать, {user?.username} ({user?.role})</h2>
        <button className="bg-secondary-container px-3 py-1.5 rounded-md" onClick={() => {
          localStorage.removeItem("token"); navigate("/login");
        }}><LogOut className="w-4 h-4" /> Выйти</button>
      </div>
      <TablesBar tablesInfo={tablesInfo} selectedTable={selectedTable} getTable={getTable} />
      {selectedTable && (
        <TableLayout
          tableName={selectedTable.name}
          columns={Object.keys(selectedTable.items[0] ?? {})}
          items={selectedTable.items}
          canWrite={selectedTable.can_write}
          canDelete={selectedTable.can_delete}
        />
      )}
    </main>
  );
}
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был успешно разработан и развернут уровень клиент-серверного приложения с организацией ролевого многопользовательского доступа к базе данных «Информационная система морских контейнерных перевозок».

Основные результаты работы:

1. Обоснован выбор архитектурного стека (Python/FastAPI, SQLAlchemy 2.0, PostgreSQL, React 19, Vite, Tailwind CSS), обеспечивающего высокую надежность, производительность и защищенность данных от атак типа SQL-инъекций и несанкционированного доступа.
2. Выполнен полный маппинг 9 реляционных отношений и 4 представлений в структуру классов ORM с сохранением ограничений целостности.
3. Спроектирована и реализована ролевая система доступа (RBAC) для трех категорий субъектов (*client*, *dispatcher*, *admin*). Доступ к чтению, модификации и удалению данных строго разграничен.
4. Реализованы сложные аналитические SQL-запросы с соединением более чем трех сущностей и группировкой (*GROUP BY*), позволяющие вычислять коммерческие показатели рейсов и сводную статистику грузопотока.
5. Создан веб-интерфейс, динамически адаптирующий доступные таблицы, формы добавления данных и панели аналитических отчетов под полномочия вошедшего пользователя.
6. Архитектура приложения полностью согласована с механизмами защиты СУБД (триггерный аудит и криптографическая защита): операции модификации вызывают срабатывание триггеров аудита в PostgreSQL, а секретные таблицы изолированы от неавторизованных ролей.